Project Overview

- **Project goal**
- Build an indoor cleaning robot system in **CoppeliaSim**
- Simulate three core capabilities:
  - **environment scanning**
  - **room traversal / coverage**
  - **obstacle avoidance**

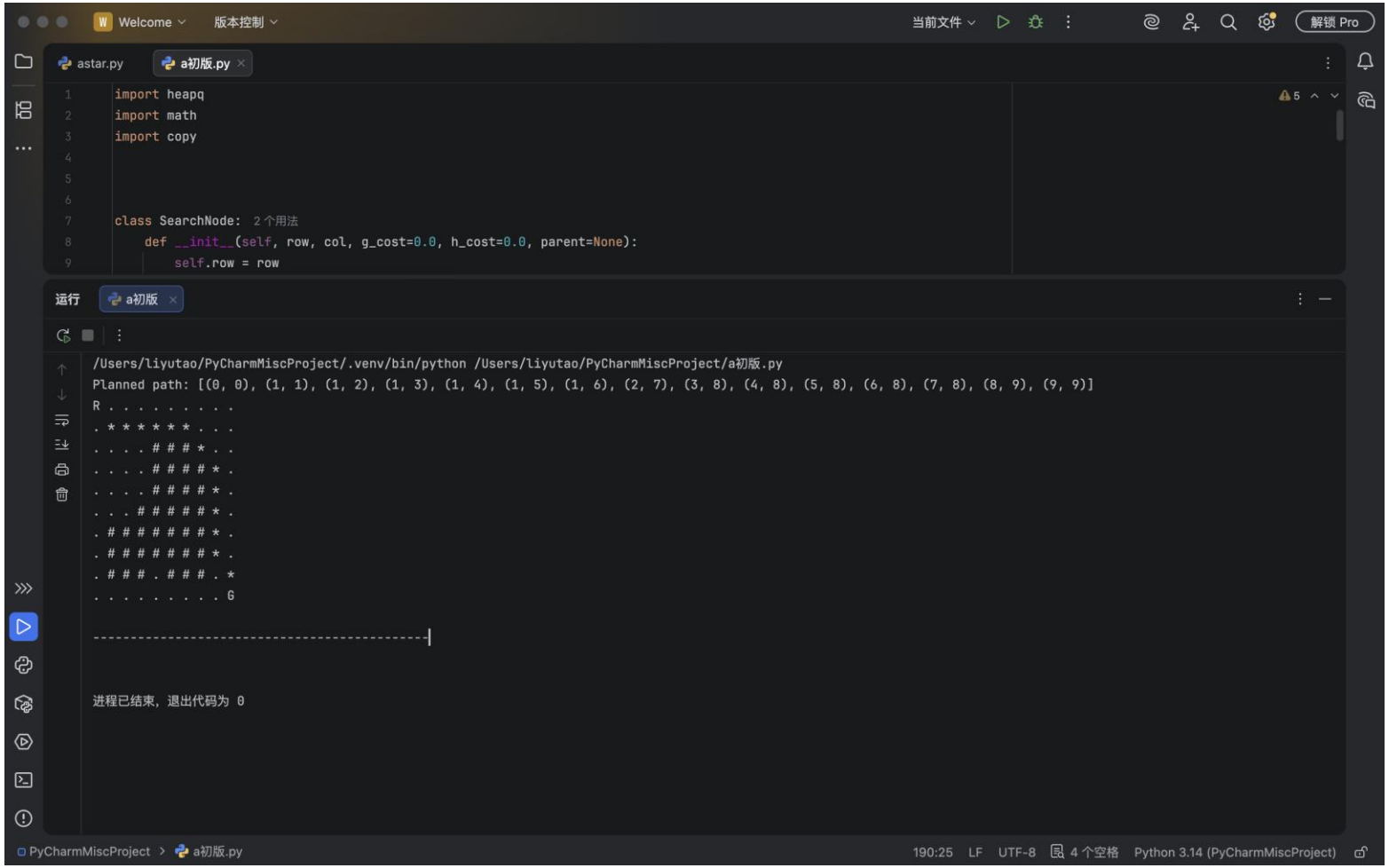
Our System Architecture

- **Main modules**
- **Scanning module**
  - detect and represent the surrounding environment
- **Traversal module**
  - cover the indoor area systematically
- **Obstacle avoidance module**
  - avoid walls, furniture, and newly appearing obstacles
- **Workflow**
- **Scanning → Traversal → Obstacle Avoidance**

Changes from A* to Coverage Traversal

- **Initial method**
- considered **A*** as the path planning method
- A* is suitable for **goal-directed navigation**
- It works well when the task is to move from one start point to one target point

- **A* Cost Function**
- f(n)=g(n)+h(n)f(n)=g(n)+h(n)**g(n)**: actual cost from robot start to current node
- **h(n)**: estimated distance from current node to target
- **f(n)**: total cost used for node selection




```python
import heapq
import math
import copy


class SearchNode:
    def __init__(self, row, col, g_cost=0.0, h_cost=0.0, parent=None):
        self.row = row
        self.col = col
        self.g_cost = g_cost
        self.h_cost = h_cost
        self.f_cost = g_cost + h_cost
        self.parent = parent

    def __lt__(self, other):
        return self.f_cost < other.f_cost

    def position(self):
        return (self.row, self.col)



class CleaningRobotPathPlanner:
    def __init__(self, environment_grid, inflation_radius=1):

        self.original_grid = copy.deepcopy(environment_grid)
        self.inflation_radius = inflation_radius
        self.safe_grid = self.inflate_obstacles(self.original_grid, inflation_radius)

    def inflate_obstacles(self, grid, inflation_radius=1):

        rows = len(grid)
        cols = len(grid[0])
        inflated_grid = copy.deepcopy(grid)

        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    for dr in range(-inflation_radius, inflation_radius + 1):
                        for dc in range(-inflation_radius, inflation_radius + 1):
                            nr, nc = r + dr, c + dc
                            if 0 <= nr < rows and 0 <= nc < cols:
                                inflated_grid[nr][nc] = 1

        return inflated_grid

    def heuristic(self, current_pos, goal_pos):

        return math.sqrt(
            (current_pos[0] - goal_pos[0]) ** 2 +
            (current_pos[1] - goal_pos[1]) ** 2
        )

    def is_free_cell(self, pos):
        r, c = pos
```

- **Problem I found**
- The updated task became an **indoor cleaning coverage task**
- A cleaning robot does not only move to one goal
- It needs to **cover the room systematically**
- **Decision**
- Changing from **goal-based planning** to **coverage traversal**
- Adopting **nearest neighbour traversal** as the main direction
- This method is more suitable for selecting the next reachable unvisited area during room coverage

My Static Nearest-Neighbour Traversal Version

- **Static logic**
- Use **nearest neighbour** to select the next unvisited reachable cell
- Use **8-neighbor search** to find candidate next cells
- If no neighbor is available, use **backtracking**

***Nearest Neighbour Traversal:**

$$x_{k+1} = \arg\min_{x \in U} d(x_k, x)$$

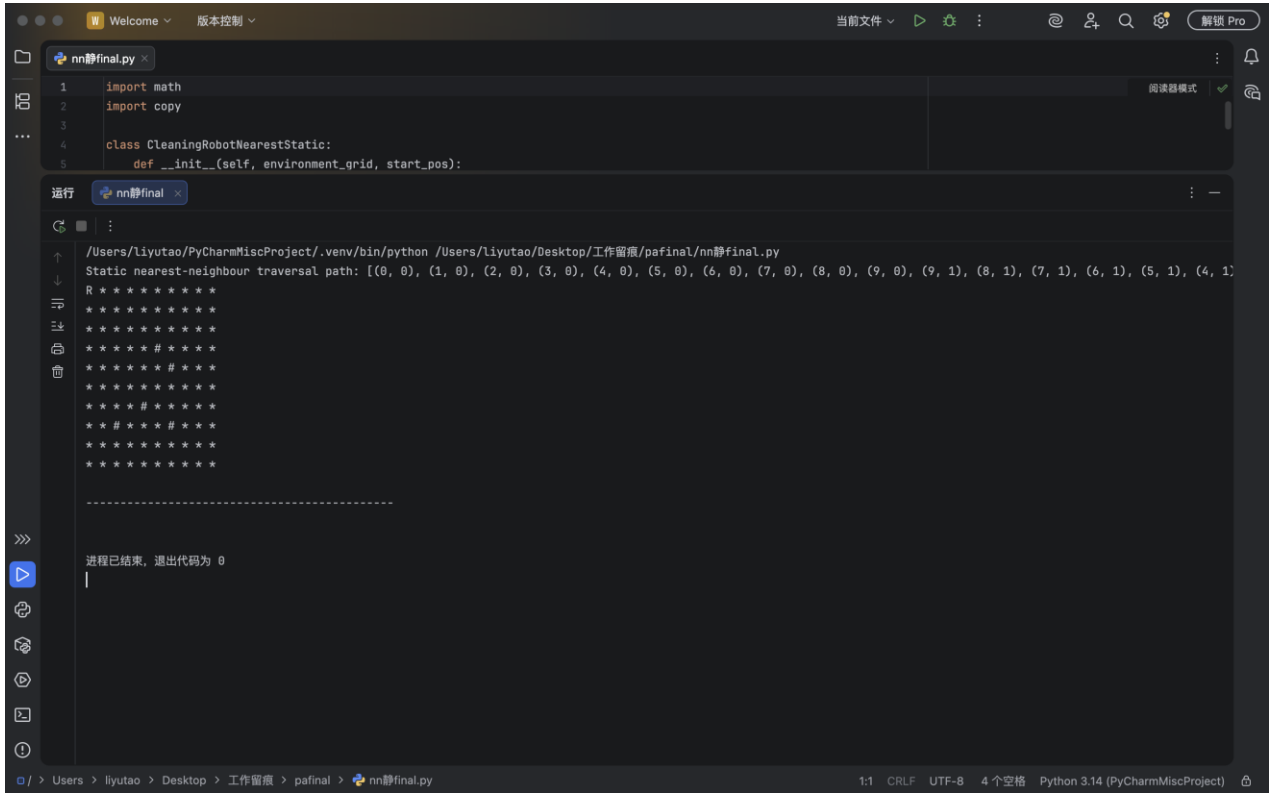
The robot selects the nearest unvisited cell at each step.

What this version can do

- Traverse the room systematically
- Avoid fixed obstacles such as walls and furniture

```
if __name__ == "__main__":
    indoor_grid = [
        [0,0,0,0,0,0,0,0,0,0],
        [0,0,0,0,0,0,0,0,0,0],
        [0,0,0,0,0,0,0,0,0,0],
        [0,0,0,0,0,1,0,0,0,0],
        [0,0,0,0,0,0,1,0,0,0],
        [0,0,0,0,0,0,0,0,0,0],
        [0,0,0,0,1,0,0,0,0,0],
        [0,0,1,0,0,0,1,0,0,0],
        [0,0,0,0,0,0,0,0,0,0],
        [0,0,0,0,0,0,0,0,0,0]
    ]

    start = (0, 0)
    robot = CleaningRobotNearestStatic(indoor_grid, start)
    path = robot.cover_area()
    print("Static nearest-neighbour traversal path:", path)
    robot.print_grid_with_path()
```

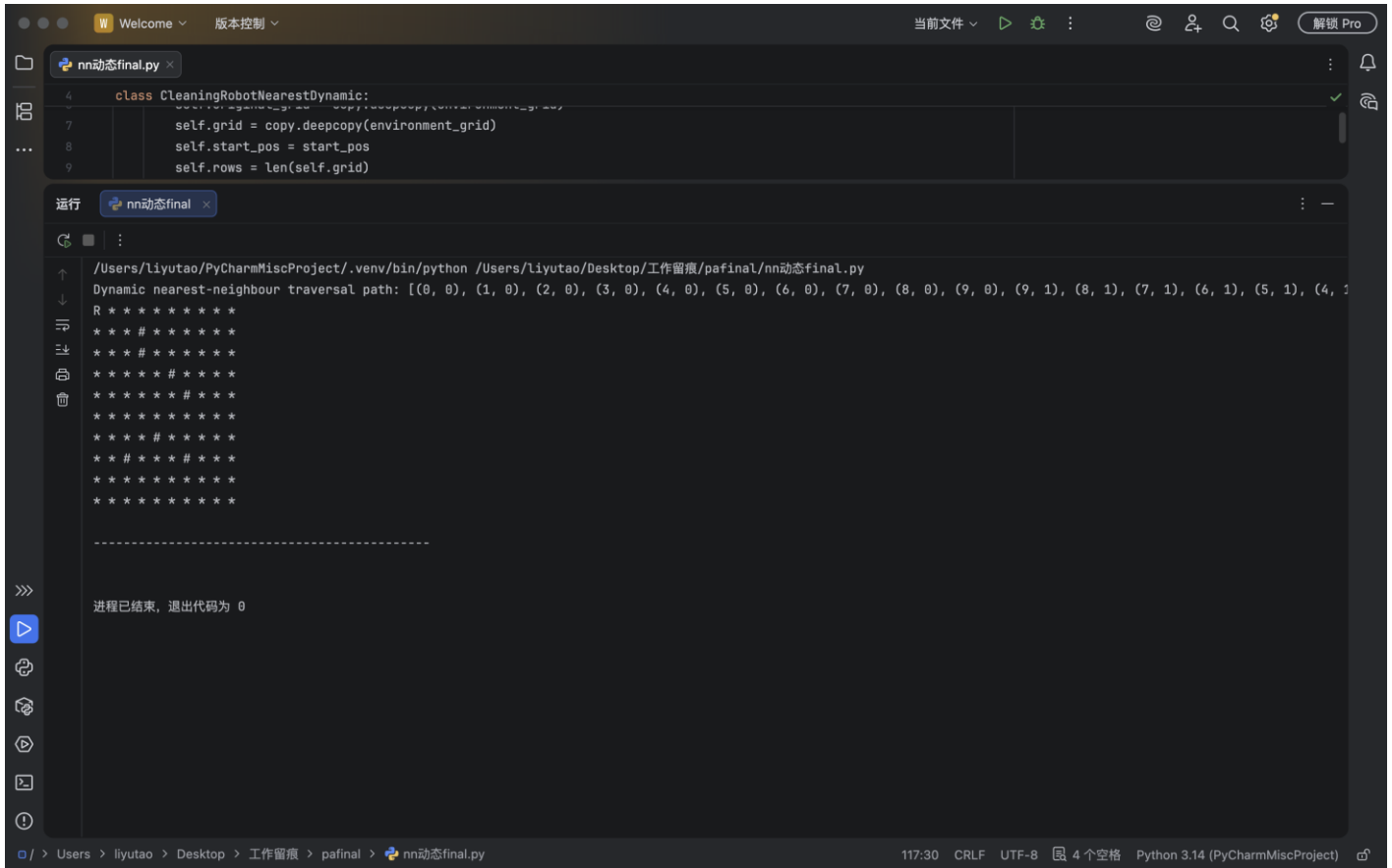


My dynamic extension

- keep the static nearest-neighbour structure
- add **obstacle update during execution**
- add **next-step validity check**
- if the next cell becomes blocked:
  - skip it
  - avoid it
  - or change traversal decision

```python
if __name__ == "__main__":
    indoor_grid = [
        [0,0,0,0,0,0,0,0,0,0],
        [0,0,0,0,0,0,0,0,0,0],
        [0,0,0,0,0,0,0,0,0,0],
        [0,0,0,0,0,1,0,0,0,0],
        [0,0,0,0,0,0,1,0,0,0],
        [0,0,0,0,0,0,0,0,0,0],
        [0,0,0,0,1,0,0,0,0,0],
        [0,0,1,0,0,0,1,0,0,0],
        [0,0,0,0,0,0,0,0,0,0],
        [0,0,0,0,0,0,0,0,0,0]
    ]

    start = (0, 0)
    robot = CleaningRobotNearestDynamic(indoor_grid, start)
    path = robot.cover_area_dynamic()
    print("Dynamic nearest-neighbour traversal path:", path)
    robot.print_grid_with_path()
```



FINAL PATHPLANNING



Limitation of scan: due to limited lateral view of sensor, the robot scan while turning around.

Prior to drive straight.

Comparison with other cleaning Project

cleaning_robot_path_planning
(https://github.com/BarkinAlper/cleaning_robot_path_planning)

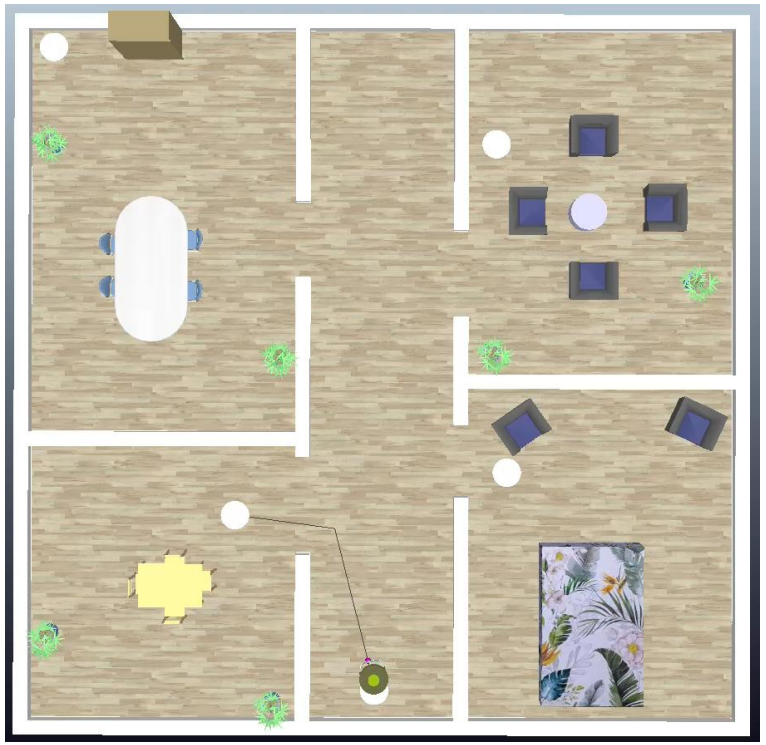
- **Similarities**
- Both projects are based on **CoppeliaSim**
- Both focus on **indoor cleaning robot behavior**
- Both include **navigation / movement** and **obstacle avoidance**

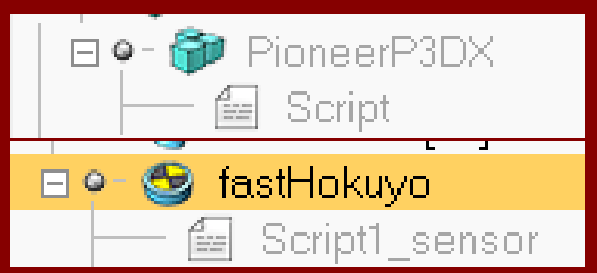
• Difference &improvemnet

Coppelia command only



Shortest path planning between waypoints

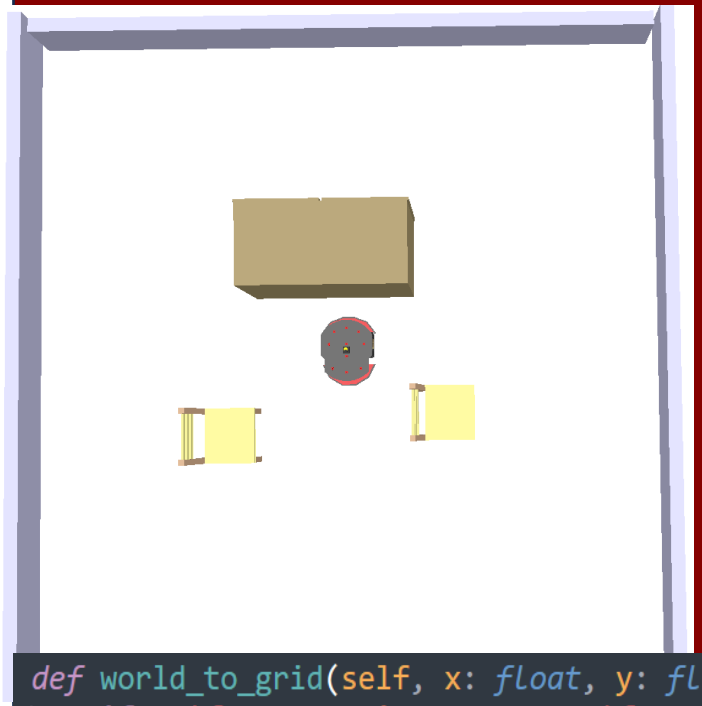




AND



Python code intergration



Tranversal cleaning through whole room

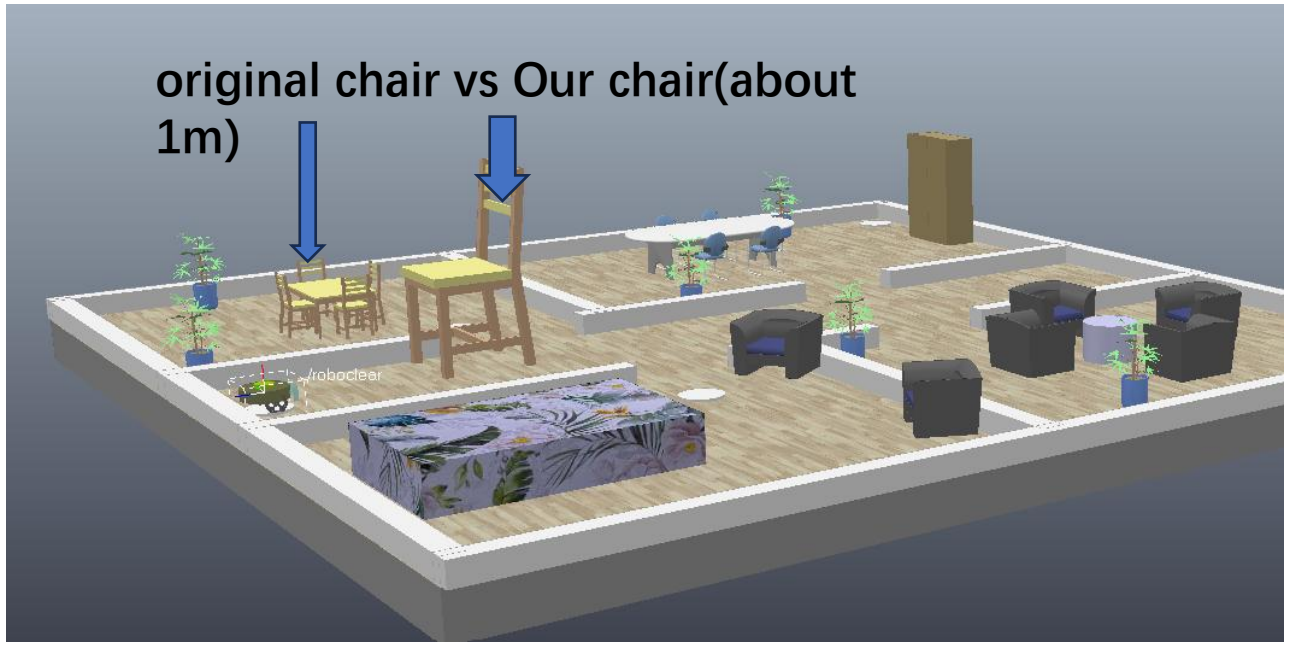
Grid map(not visualized yet)

```
def world_to_grid(self, x: float, y: float) -> Optional[Tuple[int, int]]:
    if self.start_x is None or self.start_y is None:
        return None

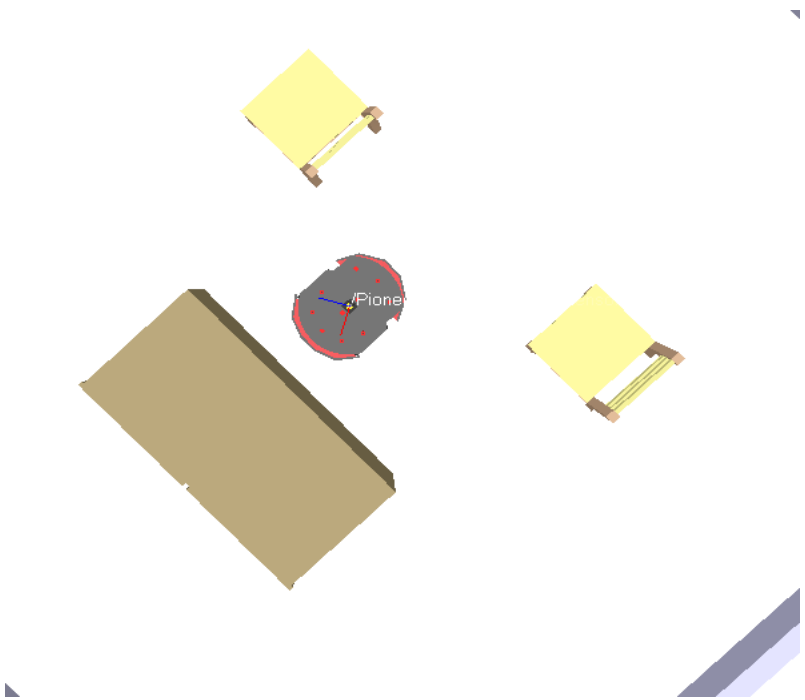
    gx = int(round((x - self.start_x) / self.resolution)) + self.grid_origin_i
    gy = int(round((y - self.start_y) / self.resolution)) + self.grid_origin_j
```

Our differences / improvements

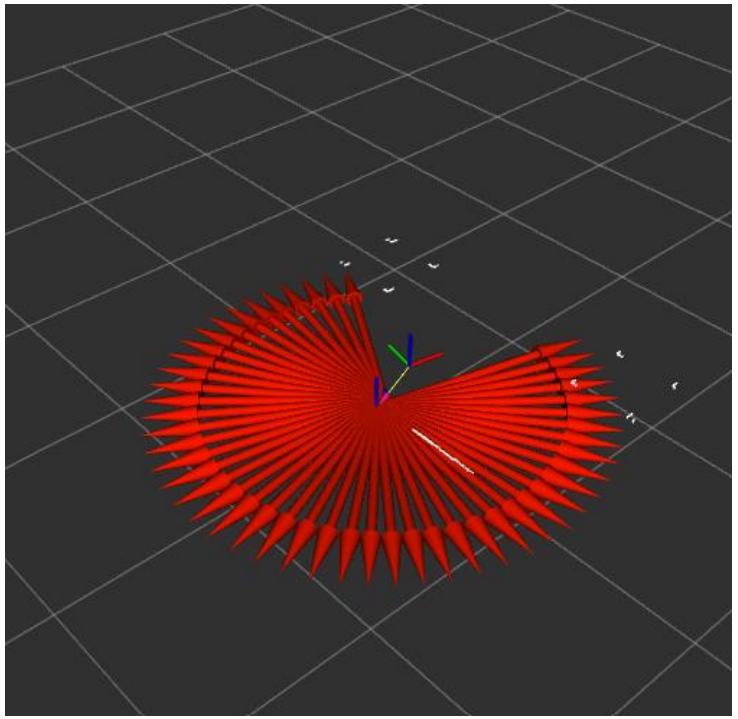
- Our project seperates control and algorithm part as python codes
- We covert the main purpose from **shortest path** to **tranversal cleaning**
- We consider and simulate the working environment in **more realistic** scale
- We also consider the extension from **static obstacle** handling to **dynamic obstacle(low speed)** response



SCAN and Sensor



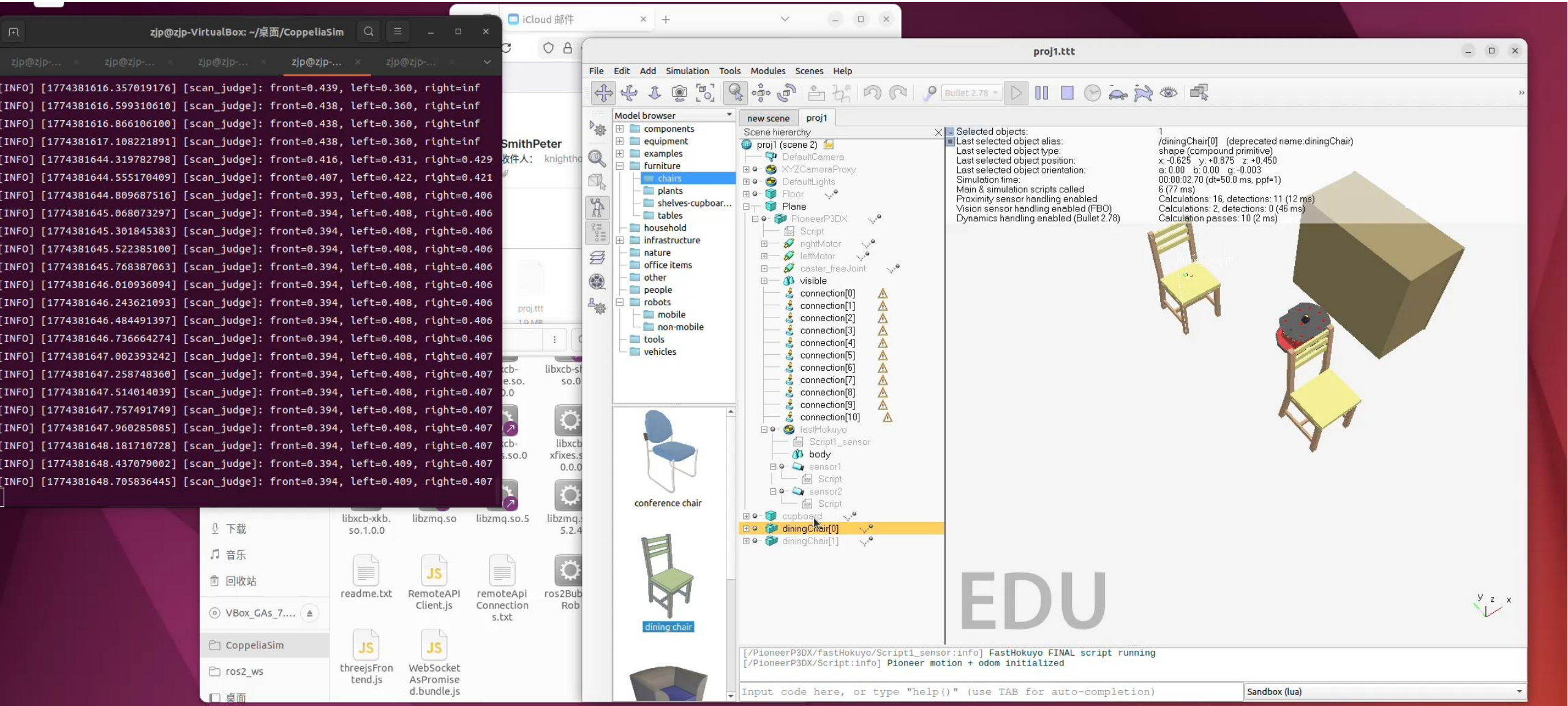
Simulation scene



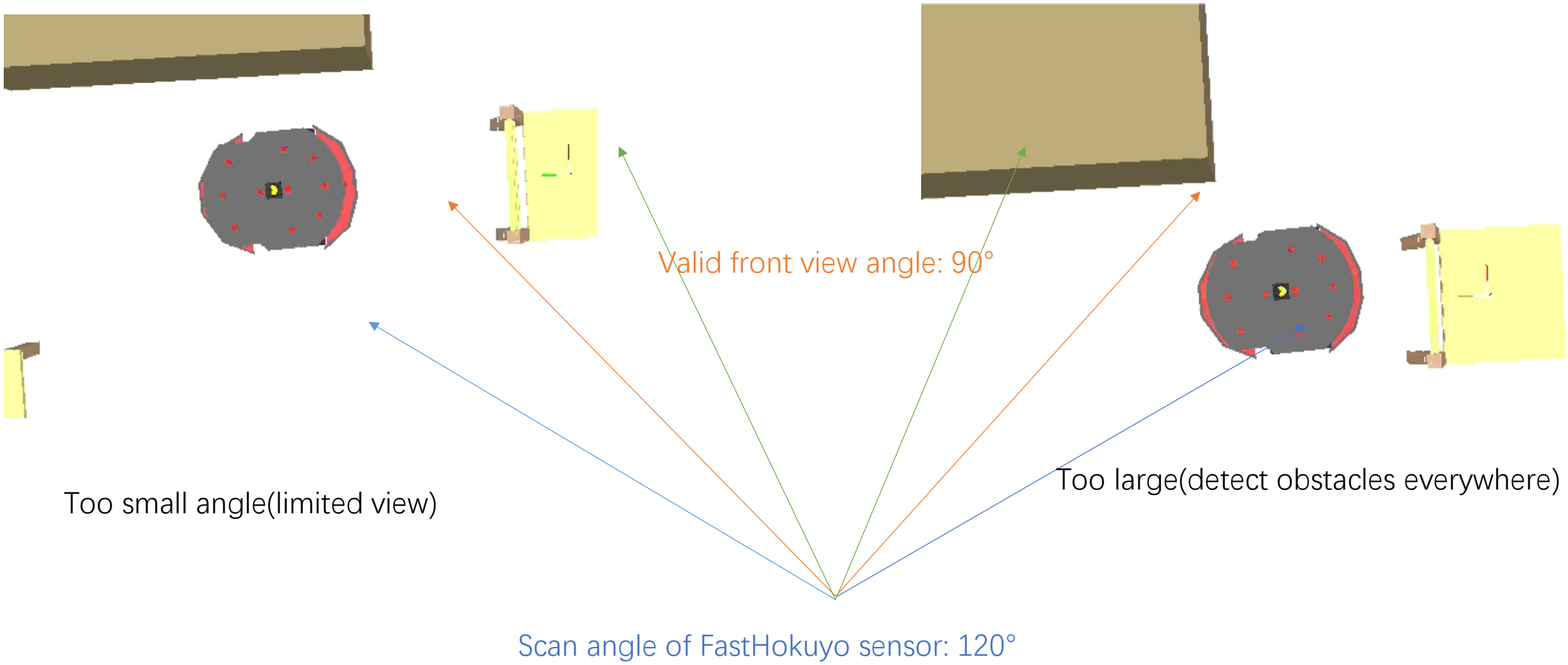
Rviz Output

Rviz could combine two sensor’s scanning data smoothly without propagation delay.
But we have to estimate and consider uncertainty due to location&angle difference of 2 sensors.

Obstacle detection&avoidance



Two sensors doesn’t well-combined their views as expected, and distance data for later not accurate enough



Manhattan distance and grid map in python

Grid map:
Use the Cartesian coordinate to make grids and
Calculate the Manhattan distance of 2 locations

```python
    def manhattan_distance(self, a: Tuple[int, int], b: Tuple[int, int]) -> int:
        return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

Output logs of location

```
[INFO] [1774517128.536842443] [grid_step_navigator_known_free_cover]: New target free cell: (110, 109)
[INFO] [1774517128.537521842] [grid_step_navigator_known_free_cover]: PLAN_NEXT: current=(110, 110), target=(110, 109), next=(110, 109)
[INFO] [1774517137.064272866] [grid_step_navigator_known_free_cover]: Reached target free cell: (110, 109)
[INFO] [1774517137.142510096] [grid_step_navigator_known_free_cover]: New target free cell: (110, 108)
[INFO] [1774517137.143279888] [grid_step_navigator_known_free_cover]: PLAN_NEXT: current=(110, 109), target=(110, 108), next=(110, 108)
[INFO] [1774517140.958900836] [grid_step_navigator_known_free_cover]: Reached target free cell: (110, 108)
[INFO] [1774517141.039106341] [grid_step_navigator_known_free_cover]: New target free cell: (110, 107)
[INFO] [1774517141.040130560] [grid_step_navigator_known_free_cover]: PLAN_NEXT: current=(110, 108), target=(110, 107), next=(110, 107)
```

```python
def current_robot_cell(self) -> Optional[Tuple[int, int]]:
    if self.robot_x is None or self.robot_y is None:
        return None
    return self.world_to_grid(self.robot_x, self.robot_y)

def get_4_neighbors(self, gx: int, gy: int) -> List[Tuple[int, int]]:
    return [
        (gx + 1, gy),
        (gx - 1, gy),
        (gx, gy + 1),
        (gx, gy - 1),
    ]

def is_in_grid(self, gx: int, gy: int) -> bool:
    return 0 <= gx < self.grid_width and 0 <= gy < self.grid_height

def is_recently_blocked(self, gx: int, gy: int) -> bool:
    key = (gx, gy)
    if key not in self.recently_blocked_cells:
        return False
```

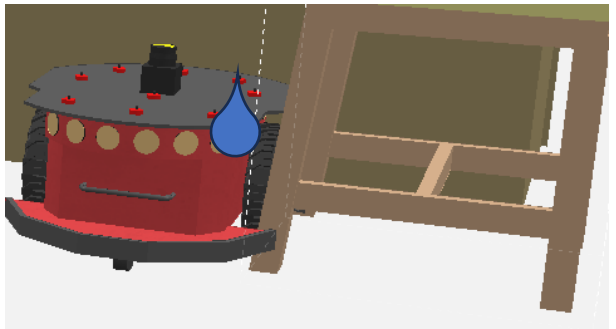
Visit adjacent grid:
note current as visit,
seek for free nodes in neighbor

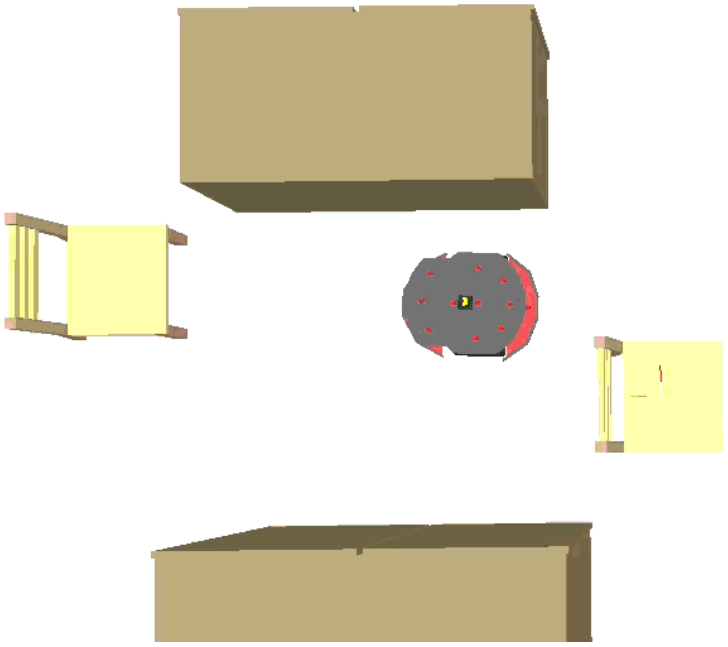
```python
def is_free_candidate(self, gx: int, gy: int) -> bool:
    if not self.is_in_grid(gx, gy):
        return False
    if self.grid[gy][gx] != self.grid_free:
        return False
    if (gx, gy) in self.visited_free_cells:
        return False
    if (gx, gy) in self.closed_free_cells:
        return False
    if self.is_recently_blocked(gx, gy):
        return False
    return True

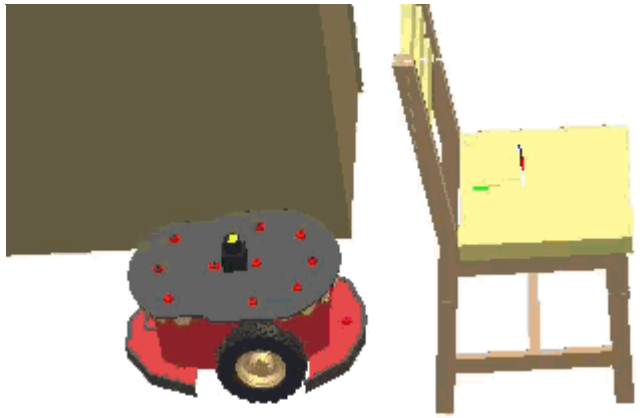
def find_unvisited_free_cells(self) -> List[Tuple[int, int]]:
    cells = []
    for gy in range(self.grid_height):
        for gx in range(self.grid_width):
            if self.is_free_candidate(gx, gy):
                cells.append((gx, gy))
    return cells

def choose_next_free_target(
    self,
    current_cell: Tuple[int, int]
) -> Optional[Tuple[int, int]]:
    candidates = self.find_unvisited_free_cells()
    if not candidates:
        return None
```

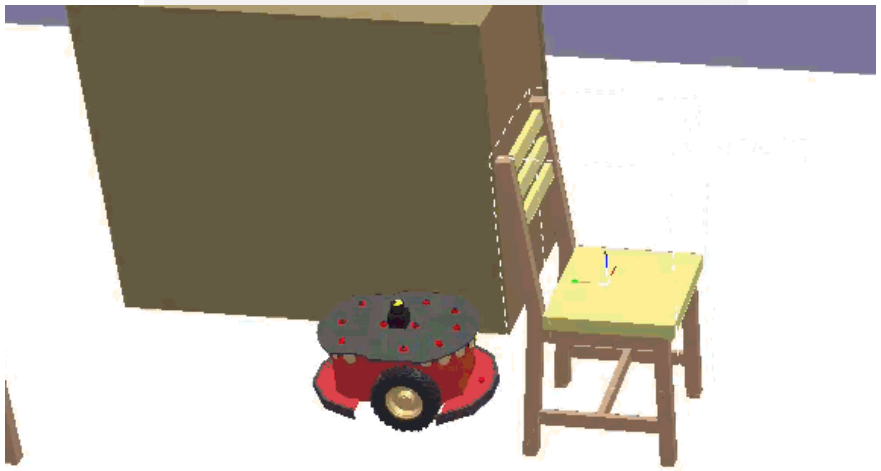
- **Problems that found**
- Previous mentioned scan data problem (**limited front** view, **lateral blind**)
- Pioneer3D's **inertia** while rotating
- Irregular shapes of Pioneer(conflit with corner of furniture)
- Sim time inconsistency







SPEED UP



NORMAL

Cops wont catch U if fast enouph, so did obstacles. –Pioneer3D

Current Limitations and Future Improvements

- **Current limitations**
- Traversal efficiency can be improved
- Coverage may still contain redundancy
- **Future improvements**
- add stronger local replanning behavior
- compare nearest neighbour with other coverage methods
- explore more systematic coverage methods for better efficiency

What we achieved

- Built a cleaning robot scenario in CoppeliaSim
- Implemented the three main functions:
  - scanning
  - traversal
  - obstacle avoidance
- Developed a traversal strategy that is more suitable for cleaning coverage
- Extended the basic traversal logic toward dynamic obstacle response

Reflection

- Compared with the background project, our project puts more emphasis on:
  - environment scanning
  - systematic room coverage
  - modular cleaning behavior

Future improvement

- stronger integration between modules
- more robust dynamic response
- better traversal efficiency
- Lateral vision, and related optimal algorithm
- Localization with SLAM map

THANK YOU